

W1 AI OVERVIEW

AI: programs/agents performing tasks normally associated w/intelligent behaviour: search, games, learning, reasoning, perception, language.

Agent: perceives env via sensors, acts via actuators; rational agent selects action maximising expected performance measure.

Task environment: PEAS = Performance, Environment, Actuators, Sensors. Classify env: fully/partially observable; deterministic/stochastic; episodic/sequential; static/dynamic; discrete/continuous; single/multi-agent.

Problem formulation: initial state, actions/successor, transition model, goal test, path cost. Abstraction hides irrelevant detail but must preserve solutions.

Search problem answer pattern: define states/actions/cost/goal, then justify algorithm by completeness, optimality, time, space.

AI views: acting humanly (Turing), thinking humanly (cognitive model), thinking rationally (logic), acting rationally (agent).

Exam cue: "choose algorithm" = state assumptions: edge costs? finiteness branching? heuristic admissible/consistent? memory limits? optimality needed?

Env property	Meaning	Risk
partial obs	hidden state	belief/probability
stochastic	uncertain outcome	expectation
sequential	future affected	planning
multi-agent	strategic	game/search

Mini ex: route planning state=city, action=drive edge, cost=distance/time, goal=destination; performance may penalise tolls/traffic.

Learning taxonomy: supervised uses labelled examples; unsupervised finds structure; reinforcement learns via reward. COMP3308 focuses search/game + supervised/unsupervised/probabilistic reasoning. AI project loop: formulate → choose representation → algorithm → train/search/infer → evaluate → analyse failure.

History/state-of-art: expert systems brittle; ML/DL data-driven success but needs data, compute, evaluation, fairness/privacy awareness.

Model vs algorithm: model is learned/constructed representation; algorithm is procedure to search/train/infer.

When explaining limitations: mention assumptions, scalability, uncertainty/noise, interpretability, data bias, and generalisation.

W2 SEARCH BASICS

Tree/graph search: frontier holds generated unexpanded nodes; explored set avoids repeated-state loops in graph search. Node contains state, parent, action, path cost g , depth.

BFS: FIFO frontier; complete if finite branching; optimal for unit costs; time/space $O(b^{d+1})$. **DFS:** stack/LIFO; low memory $O(b \cdot m)$ but not complete in infinite-depth; not optimal.

UCS: priority by $g(n)$; complete-optimal when step costs $\geq \epsilon > 0$; can expand many cheap detours. **DLS/IDS:** depth limit / repeated DLS; IDS complete-optimal for unit costs, space $O(b \cdot d)$.

Algorithm selection: unit costs + shallow goal → BFS/IDS; varying positive costs + optimal path → UCS/A*. DFS only when memory tight and optimality unneeded.

Alg	Frontier	Complete	Optimal
BFS	FIFO	yes*	unit cost
DFS	stack	no inf	pos cost
UCS	min g	yes*	pos cost
IDS	DLS loop	yes*	unit cost

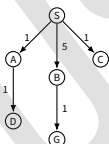
Goal test timing: on generation vs expansion can change BFS/DFS answers; follow question instruction. UCS/A* normally goal when selected for expansion for optimality.

Tie-breaking: if equal priority, apply stated order (often alphabetic/left-to-right). Write expansion order vs returned path carefully; they are different outputs.

Path reconstruction: when goal selected, follow parent links; path cost=sum edge costs; expansion order records popped/expanded nodes. Counterexample reflex: UCS may expand more nodes than BFS if a shallow expensive goal competes with many cheap non-goal nodes.

Complexity symbols: b branching factor, d shallowest goal depth, m max depth, C^* optimal cost, ϵ min positive step cost.

Repeated states: tree search can revisit; graph search prunes if better/equal path known. For UCS/A*, update/reopen if lower g found.



Common slips: confusing generated/expanded; stopping UCS when goal first generated; forgetting explored set; using BFS optimality under nonuniform costs.

W3 A* AND LOCAL SEARCH

Heuristic $h(n)$ estimates remaining cost. Greedy best-first: priority $h(n)$; fast but incomplete/Non-optimal in graph with cycles. A*: priority $f(n) = g(n) + h(n)$.

Admissible: $0 \leq h(n) \leq h^*(n)$ never overestimates. Consistent: $h(n) \leq c(n, a, n') + h(n')$ and $h(G) = 0$. Consistent → Admissible.

A* guarantees: tree search optimal if admissible positive costs; graph search optimal if consistent, or if admissible + allows reopen when better path found.

A* trace: maintain frontier sorted by f ; if the follow instruction; expand node, compute child g, h, f ; stop only when goal is popped/selected.

Search	Eval	Uses	Risk
Greedy	h	speed	bad path
UCS	g	optimal no h	many nodes
A*	$g + h$	optimal w/good h	memory
IDA*	f	found lower m_1	repeats

Dominance: if $h_2 \geq h_1$ and both admissible, h_2 is more informed and often expands no more nodes, but path cost remains optimal. Invent admissible heuristics by relaxing constraints: remove obstacles, ignore resource limits, straight-line distance, MST/assignment lower bound.

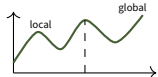
Local search: keeps current state(s), seeks good solution when path irrelevant. Hill climbing can get stuck at local maxima/plateaus/ridges. Random restart helps.

Simulated annealing: accept worse move ΔE with probability $e^{-\Delta E/T}$. High T explores/random; low T greedy/freeszes. Cooling schedule matters.

Beam/genetic: keep k states / population; selection+mutation+crossover can explore large combinatorial spaces but lacks optimality guarantees.

Mini ex: if $h(S) = 8$ & cheapest path $S \rightarrow G$ costs 6, h not admissible; if $h(A) > c(A, B) + h(B)$, not consistent.

Recall hot spot: 24/25 papers emphasise A* admissible/consistent and expansion order; write g, h, f table to avoid arithmetic slips.



W4 GAME PLAYING

Games: multi-agent, adversarial, often deterministic/perfect-information; model as game tree with MAX/MIN nodes, terminal utility, evaluation function for cutoff leaves.

Minimax: terminal utilities propagate upward: MAX takes max child, MIN takes min child. Assumes rational opponent and zero-sum preferences.

Alpha-beta pruning returns same minimax value as full minimax; can avoid evaluating branches that cannot affect final choice.

Alpha-beta: α = best value MAX can force so far; β = best value MIN can force so far. At MAX update α ; at MIN update β ; prune when $\alpha \geq \beta$.

Move ordering: best-first ordering increases pruning; perfect ordering reduces effective complexity from $O(b^m)$ toward $O(b^{m/2})$; poor ordering may prune little.

Node	Update	Prune condition
MAX	$\alpha = \max(\alpha, v)$	$\alpha \geq \beta$
MIN	$\beta = \min(\beta, v)$	$\beta \leq \alpha$
leaf	return eval	none
chance	expectation	expectimax

Trace format: write leaf values, internal backed-up values, chosen move, pruned edges. Children order is usually left-to-right unless stated.

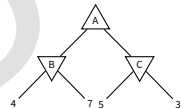
Cutoff eval: depth-limited search uses heuristic evaluation; **horizon effect:** important event just beyond cutoff hidden. Quiescence search extends unstable positions.

Chance/imperfect info: expectiminimax uses expected utilities at chance nodes; belief/opponent modelling needed when state hidden.

Mini ex: MAX root children values $B = \min(7, 8) = 7$, $C = \min(5, 3) = 3$, root=MAX=7, choose B.

Pruning intuition: at MIN, if a child value is already $\leq$ root/ancestor α , MAX would avoid this branch, so remaining siblings under MIN irrelevant.

Game traits: branching factor huge, exact solution infeasible; eval fn estimates utility from features; time limits matter more than proof of optimality.



W5 ML KNN RULES

ML: learn a target function/model from data. Supervised classification predicts discrete class; regression predicts numeric; unsupervised discovers clusters/patterns.

ML workflow: collect/clean → split train/validation/test → preprocess fit on train only → train → tune on validation/CV → final test once → interpret errors.

Never test data influence preprocessing, feature selection, hyperparameter tuning, or model choice; that is leakage.

KNN: lazy, instance-based; classify by majority/weighted vote among k nearest examples under distance metric. Training cheap, prediction slow/storage-heavy.

Distance: Euclidean $\sqrt{\sum (a_i - b_i)^2}$, Manhattan $\sum |a_i - b_i|$, Hamming for binary/categorical. Normalise numeric features first.

k	Effect	Risk
small	complex boundary	noise/high var
large	smooth boundary	bias/minority loss
odd	fewer ties	binary only
weighted	close distances matter	zero dist case

Curse dim: distances concentrate; irrelevant features harm KNN. Use feature selection/PCA/normalisation; KNN weak on high-dimensional sparse records.

Rule learning/1R: if condition THEN class; 1R chooses one attribute with lowest error and creates one rule per value. Simple, interpretable baseline.

Rules: coverage = examples matching antecedent; accuracy/confidence = correct covered/covered. Specific rules may overfit; prune conditions using validation.

Mini ex: two nearest classes yes/no at distances 1.3. Weighted vote yes: $1/1^2 = 1$, no: $1/3^2 = .11$, predict yes even if unweighted tie.

Model selection: choose by validation/CV, not training accuracy. For imbalanced data, inspect recall/precision/F1/confusion matrix.

Interpretability: KNN can cite neighbours but global rule weak; 1R/decision tree easier for "why was decision made?" cases.

Exam wording: "large scale prediction" hurts KNN; "transparent bank/medical decision" often favours decision trees/rules unless performance tradeoff stated.

Mini answer template: identify task (classification/regression), data type/scale, cost of errors, interpretability need, then recommend algorithm + metric.

W6 NB AND EVALUATION

Bayes: $P(C|x) = P(x|C)P(C)/P(x)$; compare scores $P(C)/\prod_i P(x_i|C)$.

Naïve Bayes: assumes attrs conditionally independent given class. Fast, robust for small/high-dimensional data; independence often false but useful.

NB steps: count priors $P(C)$; count conditional likelihoods $P(x_j = v|C)$; multiply for each class; choose max; optionally normalise by sum scores.

Zero probability: unseen value under class makes score 0; Laplace/add-1 smoothing: $P(v|C) = \frac{\text{count} + 1}{n_C + 1}$ for m values.

Numeric attrs: discrete or Gaussian NB with mean/variance per class; missing values can be ignored for that feature or handled as category if meaningful.

Metric	Formula	Use
Acc	$(TP + TN)/N$	balanced
Prec	$TP/(TP + FP)$	FP costly
Recall	$TP/(TP + FN)$	FN costly
F_1	$2PR/(P + R)$	tradeoff

Confusion matrix labels matter: define positive class first. Medical high-risk false negative usually more serious → prioritise recall/sensitivity.

Cross-validation: split into k folds, train k times; stratify for class imbalance; nested CV if tuning+estimating performance.

Comparing classifiers: use same folds/test set; report mean/std; consider statistical tests; include runtime, interpretability, update cost, robustness.

Mini NB: score(yes) = $P(\text{yes})P(\text{sciency}|\text{yes})P(\text{mobile}|\text{yes})$; denominator $P(x)$ cancels for argmax.

Thresholds: lowering positive threshold increases recall but may lower precision; accuracy can mislead under imbalance.

ROC/PR: ROC plots TPR vs FPR; PR better when positive rare; AUC random $\approx .5$.

Error analysis: inspect FP/FN examples, class imbalance, feature scaling, missing/noisy labels, leakage, drift.

Bias/variance: underfit = high train+val error; overfit = low train high val. Regularisation/pruning/less complexity/data augmentation help.

W7 DECISION TREES

Decision tree: internal node tests attr; branch value/range, leaf class. Prediction follows root-to-leaf path; each path yields IF-THEN rule.

Entropy $H(S) = -\sum_i p_i \log_2 p_i$; information gain $IG(S, A) = H(S) - \sum_{v \in V} \frac{|S_v|}{|S|} H(S_v)$. Pick max IG.

ID3-like build: if pure return leaf; if no attr return majority; else choose attr w/max gain; split; recurse; handle empty branch by parent majority.

Impurity: entropy and Gini $1 - \sum p_i^2$ measure class mixture; 0 = pure. Gain ratio reduces bias toward many-valued attrs.

Issue	DT handling	Trap
numeric	threshold split	sort candidates
missing	surrogate/prob split	drop data?
noise	pruning	overfit
costly attr	cost-sensitive gain	not pure IG

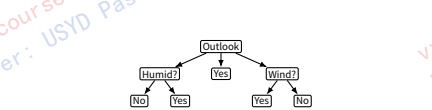
Computation: for attr value subset S_v , compute class proportions, entropy, weighted average. Show table of branch counts to earn marks.

Mini ex: class counts high=2, low=4: $H = -(2/6) \log_2 (2/6) - (4/6) \log_2 (4/6) = 0.918$.

Pruning: pre-prune stops early (min samples, max depth, min gain); post-prune grows full tree then removes branches using validation/error estimate.

Pros: interpretable, handles mixed attrs, nonlinear boundaries, little scaling. **Cons:** high variance, greedy splits, sensitive to noise/small changes.

Decision boundary: axis-aligned splits in feature space; MLP/SVM can form smoother/oblique/nonlinear boundaries.



Past-paper hot spot: entropy+IG numeric table; final answer must show branch entropy and weighted conditional entropy, not just selected attr.

W8 PERCEPTRON NN

Neuron: net input $n = w \cdot x + b$, activation/output $a = f(n)$. Step perceptron gives 1 if $n \geq 0$ else 0; boundary $w \cdot x + b = 0$.

Perceptron update: $e = t - y$; $w_i \leftarrow w_i + \eta e x_i$; $b \leftarrow b + \eta e$. Converges only if data linearly separable.

Perceptron trace: compute n , output y , error e , update each weight+bias. If correct ($e = 0$), no update.

Capability: single perceptron solves linearly separable problems; cannot solve XOR. MLP w/nonlinear hidden units can form complex boundaries.

Backprop: differentiable activations required; step function not suitable. Forward pass computes activations; backward pass uses chain rule to compute gradients; update by GD/SGD.

Activation	Derivative	Cue	Use/risk
sigmoid	saturation	prob/vanish	
tanh	zero-centred	vanish	
ReLU	0/1	dead units	
softmax	class probs	multi-class	

All-zero hidden weights make hidden neurons symmetric/identical; random initialisation breaks symmetry.

Hyperparams: hidden neurons/layers, learning rate, batch size, epochs, regularisation, activation, optimiser. Tune by validation/CV.

Training diagnosis: train loss high+val high = underfit; train low+val high = overfit; oscillation/divergence = learning rate too large; slow = too small.

Classification after training: forward propagation only; weights not changed during prediction. Backprop is training.



Mini update: $w = (.4, -.1)$, $b = -.2$, $x = (.6, .8)$, $t = 0$, $\eta = .5$: $n = .36$, $y = 1$, $e = -1$, $w' = (1.1, -.5)$, $b' = -.3$.

W9 MLP AND DEEP

MLP: layers of neurons; with nonlinear activations approximates nonlinear functions. Hidden units learn intermediate representations; depth composes features.

Backprop detail: forward $z^l = W^l a^{l-1} + b^l$, $a^l = f(z^l)$; loss L ; backward deltas; update $W \leftarrow W - \eta \nabla W \cdot L$.

No nonlinear activation → stacked linear layers collapse to one linear model; cannot learn XOR/nonlinear boundaries.

Optimisation: batch GD stable but slow; SGD noisy may escape shallow minima; mini-batch standard. Momentum/Adam adapt updates.

Nonconvex loss may converge to different weights. **Deep learning:** many-layer NN, useful for complex data + large datasets + compute. Learns features rather than manual engineering.

Regularisation: early stopping, weight decay/L2, dropout, data augmentation, smaller network, batch normalisation.

Symptom	Likely cause	Fix
val loss up	overfit	dropout/early stop
both high	underfit	more capacity/features
NaN/diverge	LR high	lower LR/scale
slow	LR low	schedule/Adam

Autocoder: network trained to reconstruct input; bottleneck learns compact representation. Can pretrain/denoise/dim-reduce, then fine-tune classifier.

CNN: convolution filters shared over spatial positions; feature map size for valid conv $= (N - A + 1) \times (M - B + 1)$ per filter for $N \times M$ image and $A \times B$ filter.

Convolution value = elementwise multiply patch and filter, then sum (+bias). Multiple filters produce multiple feature maps/channels.

CNN benefits: local connectivity, weight sharing, translation equivariance; pooling reduces spatial size and sensitivity.

Exam hot spot: compute first row of feature map; show each patch-dot-filter calculation, not just dimensions.

Gradient caveats: sigmoid/tanh saturation causes vanishing gradients; ReLU/residual/normalisation can help. Overlarge model can memorise; validate.

Mini ex: 4×4 input, 2×2 valid filter, stride $1 \rightarrow 3 \times 3$ feature map; with 6 filters $\Rightarrow$ 54 neurons.

W10 SVM ENSEMBLES

SVM: finds separating hyperplane $w \cdot x + b = 0$ with maximum margin; support vectors determine boundary. Prediction sign $(w \cdot x + b)$.

Hard-margin objective: $\min \frac{1}{2} \|w\|^2$ s.t. $y_i(w \cdot x_i + b) \geq 1$; margin width $2/\|w\|$.

Soft margin: slack variables allow violations; C trades off margin vs errors. Large C punishes errors/narrower margin; small C wider margin/more tolerance.

Kernel trick: compute inner products in feature space without explicit mapping: linear, polynomial, RBF. Kernel does not guarantee separability; it changes representation.

Perceptron vs SVM: perceptron finds any separator and may not converge if nonseparable; SVM solves convex max-margin objective and soft-margin handles violations.

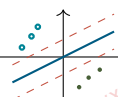
Ensemble	Mechanism	Effect
bagging	bootstrap parallel	variance down
RF	bagging+rand feats	decorrelate trees
boosting	sequential reweight	bias/error down
AdaBoost	focus mistakes	noise sensitive

Random forest: many unpurged-ish trees; each trained on bootstrap sample; each split considers random subset of features; vote/average. OOB estimates error.

Boosting: train weak learners sequentially; increase weight of misclassified examples; final weighted vote. Can reduce training error but overfits noisy/outlier labels.

AdaBoost: if weighted error $\epsilon < .5$, learner weight $\alpha = \frac{1}{2} \ln \frac{1-\epsilon}{\epsilon} > 0$; misclassified weights increase after normalisation.

Interpretability: RF more accurate/robust than one tree but less transparent; feature importance helps but can be biased. Single tree/rules preferable for strict explanations.



Past-paper trap: "kernel trick ensures separability" is false; "linear SVM always separates arbitrary labels" false; soft margin compromises.

W11 PAST-PAPER WORKS

Probability basics: product rule: $P(x_1, \dots, x_n) = P(x_1) \prod_{i=2}^n P(x_i | x_1, \dots, x_{i-1})$; total probability $P(x) = \sum_y P(y) P(x|y)$; Bayes' theorem $P(y|x) = \frac{P(x|y)P(y)}{\sum_z P(x|z)P(z)}$.

Bayesian network: DAG where nodes are variables and directed edges are factorises as $P(X_1, \dots, X_n) = \prod_i P(X_i | \text{parents}(X_i))$. Joint query: instantiate factorises over hidden variables. Enumerate conditional independence. Used given parents. Use d-separation: colliders blocked unless co-occur.



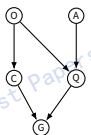
Path	Form	Observed middle
chain	$A \rightarrow B \rightarrow C$	blocks
fork	$A \rightarrow B \leftarrow C$	blocks
collider	$A \rightarrow B \leftarrow C$	opens

Collider trap: observing effect/explaining-away variable makes causes dependent; observing a chain/fork middle blocks information flow.

Variable elimination: choose hidden var, multiply factors containing it, sum it out, continue; efficiency depends on ordering/treewidth.

CPT learning: estimate conditional probabilities from counts for each parent configuration; smoothing avoids zero probabilities. Naïve Bayes is BN with class parent of features.

Exam expression: $P(q|c, g) = P(q|c, g)P(c|g)$; expand numerator by summing hidden parents/variables using factorisation.



Mini factorisation for graph above: $P(O, A, C, G, Q) = P(O)P(A)P(C|O)P(Q|O, A)P(G|C, Q)$.

Recall hot spot: conditional independence given evidence + numeric probability from CPT table; draw active paths before answering yes/no.

W12 UNSUPERVISED LEARNING

Clustering: partition data so within-cluster similarity high, between-cluster similarity low; no labels. Similarity usually via distance.

K-means: choose k centroids; assign each point to nearest centroid; update centroid to cluster mean; repeat until stable. Minimises within-cluster SSE locally, not globally.

K-means limitations: needs k , sensitive to initial centroids/outliers/scaling, assumes roughly spherical/equal-size convex clusters, poor on elongated/nonconvex clusters.

Choosing k : elbow plot, silhouette, domain constraints, validation utility; increasing k is not always meaningful and can overfit structure.

Hierarchical agglomerative: start singletons; repeatedly merge closest clusters; dendrogram; cut by threshold or number of clusters.

Linkage: single-link nearest pair (can chain elongated clusters); complete-link farthest pair (compact); average-link mean pair distance; centroid-link by centroid distance.

||
||
||